

System Conversation Logging — Implementation Plan

1. Purpose

Add a structured, leveled conversation logging system to the LangGraph agent. All agent activity — user prompts, tool calls (Genie, Vector DB), LLM reasoning, and final responses — will be optionally written to a Unity Catalog Delta table in Databricks.

The system is designed to be:

- **Non-blocking** — all writes are async fire-and-forget; never slows the agent
- **Toggleable** — a single env var controls everything (`AGENT_LOG_LEVEL`)
- **Leveled** — three modes with increasing detail
- **Zero-dependency** — uses the `WorkspaceClient` already present in `agent.py`

2. The Three Log Levels

`off` — Default (Production)

- Nothing is written to Databricks
- Zero overhead, zero latency impact
- Recommended for production deployments

`min` — Minimum (Business Audit)

- **2 rows per conversation turn** only
- Row 1: The user's question (captured at request start)
- Row 2: The final answer delivered to the user (captured at stream end)
- Includes end-to-end `duration_ms` on the final row
- Use case: Usage analytics, auditing, "what did users ask and what did they get?"

`verbose` — Full Debug (Engineering)

- **Every activity row** — full chain of thought
- Captures every tool call and its response individually
- Includes per-activity `duration_ms`
- Includes `raw_payload` (full JSON from Genie/Vector DB)
- Use case: Debugging slow responses, tracing errors, performance tuning

Rows written per level

Activity	<code>off</code>	<code>min</code>	<code>verbose</code>
User Request	✗	✓	✓
LLM Thinking	✗	✗	✓
Call Genie	✗	✗	✓
Call Vector DB	✗	✗	✓
Call Copilot	✗	✗	✓
Final Response	✗	✓	✓

3. Log Table Schema

Table name: `slf_srvc.test_db.agent_debug_log` (configurable via `DEBUG_LOG_TABLE` env var)

Column	Type	Nullable	Description
logged_at	TIMESTAMP	NOT NULL	UTC timestamp when the row was written
conversation_id	STRING	NOT NULL	Chat session UUID (from request context)
session_turn	INT	NOT NULL	Increments per user message within a conversation
user_id	STRING	NOT NULL	User email (e.g. <code>javier.delacruz@wheelsup.com</code>)
orchestrator_model	STRING	NOT NULL	LLM endpoint name (e.g. <code>databricks-claude-opus-4-7</code>)
activity	STRING	NOT NULL	User Request, LLM Thinking, Call Genie, Call Vector DB, Call Copilot, Final Response
user_prompt	STRING	NULL	The original user question
response	STRING	NULL	Human-readable response text from the activity
duration_ms	INT	NULL	Time taken for this activity in milliseconds
status	STRING	NOT NULL	success or error
error_message	STRING	NULL	Error detail if <code>status = error</code> , else null
raw_payload	STRING	NULL	Full JSON payload — only populated at verbose level

Column population by level

Column	min	verbose
logged_at	✓	✓
conversation_id	✓	✓
session_turn	✓	✓
user_id	✓	✓
orchestrator_model	✓	✓
activity	✓ (2 values only)	✓ (all values)
user_prompt	✓	✓

response	✅ (final only)	✅ (every activity)
duration_ms	✅ (end-to-end)	✅ (per activity)
status	✅	✅
error_message	✅	✅
raw_payload	❌	✅

4. Environment Variables — The Toggle

Variables

```
# _____
# AGENT CONVERSATION LOGGING
# _____

# Controls logging verbosity.
# Values: off | min | verbose
# Default: off
# - off      → nothing written, zero overhead (use in production)
# - min      → user prompt + final answer only (2 rows per turn)
# - verbose  → every tool call, LLM step, and response (full debug)
AGENT_LOG_LEVEL=off

# Unity Catalog table to write logs into.
# Must exist before logging is enabled (see Part 3 for DDL).
DEBUG_LOG_TABLE=slf_srvc.test_db.agent_debug_log

# SQL Warehouse ID used to execute INSERT statements.
# Find: Databricks UI → SQL Warehouses → your warehouse → Connection Details
DEBUG_WAREHOUSE_ID=
```

How to toggle

Goal	Set AGENT_LOG_LEVEL to
Maximum speed, no logging	off
Track what users ask + final answers	min
Full debug — see every tool call	verbose